SAPIENZA
UNIVERSITÀ DI ROMA

Faculty of Information Engineering, Informatics and Statistics
Department of Computer Science

Deep Learning and Applied AI

Author:
Simone Lidonnici

25 march 2026

Contents

1 Linear algebra **1**

1.1 Vector spaces . . . 1

1.1.1 Base of a vector space . . . 1

1.2 Linear maps . . . 2

1.2.1 Vector space of linear maps . . . 2

1.2.2 Product of linear map . . . 2

1.3 Matrices . . . 3

1.3.1 Vector represented as matrix . . . 3

1.3.2 Matrix manipulation . . . 3

1.3.3 Products with matrices . . . 4

1.3.4 Matrix parameters . . . 4

2 Deep Learning **6**

2.1 Convex function . . . 6

2.1.1 Convex functions in $\mathbb{R}^n$ . . . 7

2.2 Vector lenghts . . . 7

2.3 Linear regression . . . 7

2.3.1 Matrix notation . . . 8

2.3.2 Linear regression in $\mathbb{R}^n$ . . . 9

2.3.3 Polynomial regression . . . 9

2.4 Logistic regression . . . 10

2.5 Gradient descent . . . 10

2.5.1 Stochastic gradient descent . . . 11

2.6 Back-propagation . . . 12

2.6.1 Computational graphs . . . 13

1

Linear algebra

1.1 Vector spaces

A vector space V is a set with an addition and a scalar multiplication that respect some properties:

- **Commutativity**: $u + v = v + u \; \forall u, v \in V$ and also $u + v \in V$
- **Associativity:** $(u + v) + w = u + (v + w) \; \forall u, v, w \in V$ and $(ab)v = a(bv) \; \forall a, b \in \mathbb{R}$ and $av \in V$
- **Additive identity**: $\exists 0 | v + 0 = v \; \forall v \in V$
- **Additive inverse:** $\forall v \in V \; \exists w \in V | v + w = 0$
- **Multiplicative identity**: $\exists 1 \in \mathbb{R} | 1v = v \; \forall v \in V$
- **Distributive**: $a(u + v) = au + av \; \forall u, v \in V \; \forall a \in \mathbb{R}$

Example:
The set of all functions $f : [0, 1] \to \mathbb{R}$ with addition and scalar multiplication as:

$$(f + g)(x) = f(x) + g(x)$$

$$(\lambda f)(x) = \lambda f(x)$$

And additive identity and inverse as:

$$0(x) = 0$$

$$(-f)(x) = -f(x)$$

This is a vector space and all sets of functions $f : S \to \mathbb{R}$ is a vector space.

1.1.1 Base of a vector space

A base of a vector space V is a set of vectors $v_1, \ldots, v_n \in V$ that are linearly indipendent and spans V.
The span of a set of vectors is defined as:

$$\text{Span}(v_1, \ldots, v_n) = \{\alpha_1 v_1 + \cdots + \alpha_n v_n | \alpha_1, \ldots, \alpha_n \in \mathbb{R}\}$$

So, to span V and be linearly indipendent means that each vector $v \in V$ can be represented by only one linear combination of $v_1, \ldots, v_n$:

$$v = \sum_{i=1}^{n} \alpha_i v_i$$

A vector space has infinite bases, all with the same number of vectors and this number is the dimension of the vector space.

1.2 Linear maps

A linear map $T : V \to W$ is a function with some properties:

- **Additivity**: $T(u + v) = T(u) + T(v)\ \forall u, v \in V$
- **Homogeneity**: $T(\lambda v) = \lambda T(v)\ \forall v \in V\ \forall \lambda \in \mathbb{R}$

Example:
The equation:

$$y = ax + b$$

is not a linear map, because of the constant b, that in deep learning is called a bias.
This equation:

$$y = z \sin(x) + z^2 \sin(x)$$

is not a linear map in respect to z or x but is a linear map in respect to $\sin(x)$.

1.2.1 Vector space of linear maps

Fixed two vector spaces V, W, the set of all linear map $T : V \to W$ is itself a vector space with addition and scalar multiplication:

$$(S + T)(v) = S(v) + T(v)$$

$$(\lambda T)(v) = \lambda T(v)$$

The additive identity is the linear map that maps all values to the identity of the vector space W.

1.2.2 Product of linear map

Taken two linear maps $T : V \to W$ and $S : W \to Z$, their product is defined as $ST : V \to Z$:

$$(ST)(v) = S(T(v))$$

So, is the composition $S \circ T$. The product has different properties:

- **Associativity**: $(T_1 T_2) T_3 = T_1 (T_2 T_3)$
- **Identity**: $\exists I | TI = IT = T$
- **Distributive**: $(S_1 + S_2)T = S_1 T + S_2 T$

The product is not commutative in general.

1.3 Matrices

Let $T : V \to W$ be a linear map, $v_1, \ldots, v_n \in V$ the base of V and $w_1, \ldots, w_m \in W$ the base of W. The map T can be represented by a $m \times n$ matrix of values in $\mathbb{R}$:

$$T = \begin{vmatrix} T_{1,1} & \ldots & T_{1,n} \\ \vdots & \ddots & \vdots \\ T_{m,1} & \ldots & T_{m,n} \end{vmatrix}$$

The entries of the matrix are defined in such a way that:

$$T(v_j) = T_{1,j}w_1 + \cdots + T_{m,j}w_m$$

The matrix encodes how the vectors of the base are mapped, but it also maps all other vector in V:

$$T(v) = T(\sum_{\forall j} \alpha_j v_j) = \sum_{\forall j} T(\alpha_j v_j) = \sum_{\forall j} \alpha_j T(v_j)$$

The matrix representation of T depends on the base chosed.

1.3.1 Vector represented as matrix

In a vector space V with base $v_1, \ldots, v_n$ an arbitrary vector $v \in V$ can be defined as a $n \times 1$ matrix:

$$v = \begin{vmatrix} c_1 \\ \vdots \\ c_n \end{vmatrix}$$

In which the coefficient are such that:

$$v = c_1 v_1 + \cdots + c_n v_n$$

1.3.2 Matrix manipulation

A matrix is symmetric if:

$$A = A^T \implies A_{i,j} = A_{j,i} \; \forall i, j$$

For a product $A = BC$ the transpose and inverse are:

$$A^T = (BC)^T = C^T B^T$$

$$A^{-1} = (BC)^{-1} = C^{-1} B^{-1}$$

A matrix is orthogonal if:

$$A^{-1} = A^T$$

And when the matrix is orthogonal we have that $A^T A = I$.

1.3.3 Products with matrices

There a different ways in which matrices can be multiplied.
Matrix-Vector product:

$$Xy = \begin{vmatrix} | & \dots & | \\ x_1 & & x_n \\ | & \dots & | \end{vmatrix} \begin{vmatrix} y_1 \\ \vdots \\ y_n \end{vmatrix} = \sum_{j=1}^{n} y_j \begin{vmatrix} | \\ x_j \\ | \end{vmatrix}$$

Vector-Matrix product:

$$y^T X = (X^T y)^T$$

Matrix-Matrix product:

$$XY = \begin{vmatrix} - & x_1^T & - \\ \vdots & & \vdots \\ - & x_m^T & - \end{vmatrix} \begin{vmatrix} | & \dots & | \\ y_1 & & y_n \\ | & \dots & | \end{vmatrix} = \begin{vmatrix} | & \dots & | \\ Xy_1 & & Xy_n \\ | & \dots & | \end{vmatrix} = \begin{vmatrix} - & x_1^T Y & - \\ \vdots & & \vdots \\ - & x_m^T Y & - \end{vmatrix}$$

Vector-Vector product (inner):

$$x^T y = \alpha$$

Vector-Vector product (outer):

$$xy^T = \begin{vmatrix} x_1 \\ \vdots \\ x_m \end{vmatrix} \begin{vmatrix} y_1 & \dots & y_n \end{vmatrix} = \begin{vmatrix} | & \dots & | \\ y_1 x & & y_n x \\ | & \dots & | \end{vmatrix}$$

1.3.4 Matrix parameters

The trace of a matrix is the sum of the diagonal:

$$tr(A) = \sum_{\forall i} a_{i,i}$$

The trace is a linear map, since:

$$tr(A + B) = tr(A) + tr(B)$$

$$tr(\alpha A) = \alpha tr(A)$$

It is also invariant to transpose and cyclic permutations:

$$tr(A) = tr(A^T)$$

$$tr(ABC) = tr(CAB) = tr(BCA)$$

The gradient of the trace of a matrix is computed by partial derivatives:

$$\nabla tr(A) = \nabla \sum_{\forall i} a_{i,i} = \frac{\partial}{\partial a_{i,j}} \sum_{\forall i} a_{i,i}$$

The result is the identity matrix, because the derivatives are 1 only for an element in the diagonal like $a_{i,i}$.

The Frobenius norm of a matrix is:

$$||X||_F^2 = tr(XX^T) = tr(X^TX)$$

For a vector is much simple:

$$||x||_F^2 = tr(xx^T) = tr(x^Tx) = x^Tx$$

With this we can calculate the distance between two matrices:

$$||A - B||_F^2 = tr((A - B)^T(A - B)) = tr(A^TA) + tr(B^TB) - 2tr(A^TB)$$

The 1 vector (the vector with all 1) can be used to calculate sum of values of a matrix easy. To calculate the sum of values on each row:

$$A1$$

The sum of values of each column:

$$1^TA$$

The sum of all values:

$$1^TA1$$

Also we have that:

$$x^TAx = tr(x^TAx) = tr(xx^TA) = tr(XA)$$

In which $X = xx^T$ and creating this matrix is called lifting.
The permutation matrices are orthogonal matrices that are doubly stochastic and have a single 1 in every column and row:

$$\begin{vmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{vmatrix} \begin{vmatrix} 1 \\ 2 \\ 3 \end{vmatrix} = \begin{vmatrix} 2 \\ 1 \\ 3 \end{vmatrix}$$

The convex combination of two permutation matrices P, Q is another permutation matrix:

$$\alpha P + (1 - \alpha)Q = D \ \forall \alpha \in [0, 1]$$

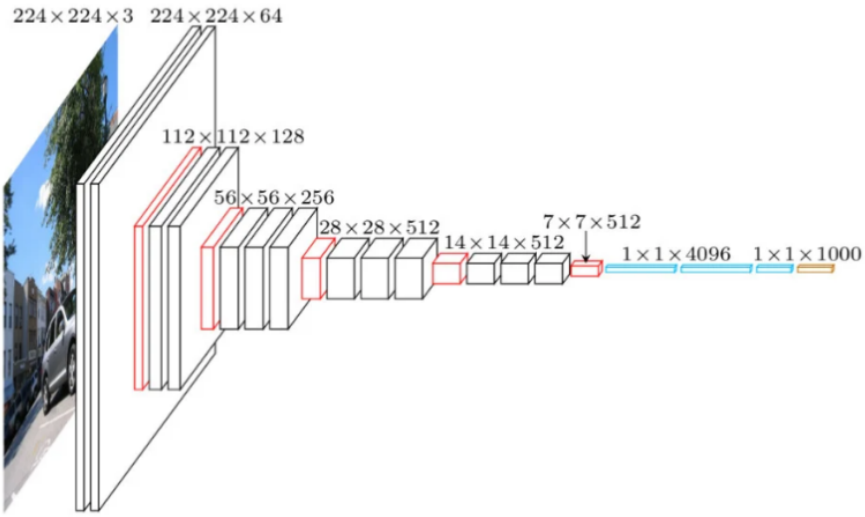
2

Deep Learning

In deep learning a model is represented by an highly parametrized model called deep neaural network. This model is a sequence of linear function with parameters Θ:

$$f_\Theta(x) = y$$

A model is a sequence of blocks, each with a predefined structure and some unknown parameters γ.



Finding the parameter values is called training and is done by minimizing a function called loss ℓ.

2.1 Convex function

A function is convex if it fullfil the **Jensen's inequality**:

$$f(\alpha x + (1-\alpha)y) \leq \alpha f(x) + (1-\alpha) f(y)$$

For all x, y and $\alpha \in (0, 1)$.
We also assume that f is differentiable, so we can compute the derivative $\frac{df}{dx}$ for all points. The global minimizer for x is where $\frac{df(x)}{dx} = 0$.

2.1.1 Convex functions in $\mathbb{R}^n$

If we have functions in $\mathbb{R}^n$ like the loss function:

$$\ell : \mathbb{R}^n \to \mathbb{R}$$

The derivative of this function is defined as the gradient, calculated with the partial derivatives on the vector of parameters Θ:

$$\nabla_{Theta}\ell(\Theta) = \begin{vmatrix} \frac{\partial \ell}{\partial \theta_1} \\ \vdots \\ \frac{\partial \ell}{\partial \theta_n} \end{vmatrix}$$

The loss function is convex:

$$\ell(\alpha x + (1-\alpha)y) \leq \alpha\ell(x) + (1-\alpha)\ell(y)$$

So, there is a global minimizer:

$$\nabla_\Theta \ell(\Theta) = 0 \implies \ell(x) \leq \ell(y)\ \forall y \in \mathbb{R}^n$$

2.2 Vector lenghts

In an high dimensional space $\mathbb{R}^n$, for two points $x, y \in \mathbb{R}^n$ the Euclidean distance is defined as:

$$||x-y||_2 = \sqrt{\sum_{i=1}^{d}(x_i - y_i)^2}$$

The Euclidean distance can be generalized by the L^p norm, that is defined as:

$$\delta_p(x,y) = \left(\sum_{i=1}^{d} ||x_i - y_i||^p\right)^{\frac{1}{p}}$$

The lenght (norm) of a vector is its distance from the distance:

$$||x||_2 = \sqrt{\sum_{i=1}^{d}(x_i)^2} = \sqrt{x^T x}$$

2.3 Linear regression

We take as example the simplest case, with the function:

$$f_\Theta(x_i) = ax_i + b = y_i$$

This is a linear model with 2 parameters $\Theta = \{a, b\}$ and data as pairs (x_i, y_i).
We need to choose the parameters a and b such that they minimize the **mean squared error (MSE)**, between input and predicted output:

$$\epsilon = \min_\Theta \ell(\{x_i, y_i\})$$

In which ℓ is the loss function:

$$\ell : \Theta \to \mathbb{R}$$

$$\ell_\Theta = \sum_{\forall i} (y_i - f_\Theta(x_i))^2$$

We want to minimize the loss in respect to the parameters Θ and not the data.
To minimize the parameters we want to find Θ^* minimize this function:

$$\Theta^* = \arg\min_{\Theta \in \mathbb{R}^2} (\ell(\Theta))$$

Where $\ell : \mathbb{R} \to \mathbb{R}$ is defined as:

$$\ell(a, b) = \sum_{\forall i} (y_i - (ax_i + b))^2$$

We find the solution by setting $\nabla_\Theta \ell(\Theta) = 0$:

$$\nabla_\Theta \sum_{\forall i} (y_i - (ax_i + b))^2 = \sum_{\forall i} \nabla_\Theta (y_i - (ax_i + b))^2 = \sum_{\forall i} \nabla_\Theta (y_i^2 + a^2 x_i^2 + b^2 - 2ax_i y_i - 2by_i + 2abx_i)$$

$$= \begin{vmatrix} \frac{\partial \ell}{\partial a} \\ \frac{\partial \ell}{\partial b} \end{vmatrix} = \begin{vmatrix} \sum_{\forall i} 2ax_i^2 - 2x_i y_i + 2bx_i \\ \sum_{\forall i} 2b - 2y_i + 2ax_i \end{vmatrix}$$

We set this 2 linear equation as 0:

$$\begin{vmatrix} \sum_{\forall i} 2ax_i^2 - 2x_i y_i + 2bx_i \\ \sum_{\forall i} 2b - 2y_i + 2ax_i \end{vmatrix} = \begin{vmatrix} 0 \\ 0 \end{vmatrix}$$

2.3.1 Matrix notation

The matrix notation of the equations $y_i = ax_i + b$ can be written in 2 ways:

$$\underbrace{\begin{vmatrix} y_1 \\ \vdots \\ y_n \end{vmatrix}}_{Y} = \underbrace{\begin{vmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{vmatrix}}_{X} \underbrace{\begin{vmatrix} a \\ b \end{vmatrix}}_{\Theta} = a \begin{vmatrix} x_1 \\ \vdots \\ x_n \end{vmatrix} + b$$

In which Y is the matrix representing all the y_i and X is the matrix representing all x_i.
So, calculating the MSE is:

$$\ell(\Theta) = ||Y - X\Theta||_2^2 = (Y - X\Theta)^T (Y - X\Theta) = Y^T Y - 2Y^T X\Theta + \Theta^T X^T X\Theta$$

$$\implies \Theta = \frac{X^T Y}{X^T X} \implies \Theta = (X^T X)^{-1} X^T Y$$

2.3.2 Linear regression in $\mathbb{R}^n$

If the data points $(x_i, y_i) \in \mathbb{R}^d$ the function representing the model is:

$$y_i = Ax_i + b$$

where A is a matrix of parameters and b is a vector of bias for every dimension. Then Θ is a matrix constructed as:

$$\Theta = \begin{vmatrix} a_{1,1} & \dots & a_{1,d} & b_1 \\ \vdots & \ddots & \vdots & \vdots \\ a_{1,d} & \dots & a_{d,d} & b_d \end{vmatrix}$$

If we define the matrices X, Y as:

$$X = \begin{vmatrix} | & \dots & | \\ x_1 & & x_n \\ | & & | \\ 1 & \dots & 1 \end{vmatrix} \qquad Y = \begin{vmatrix} | & \dots & | \\ y_1 & & y_n \\ | & & | \end{vmatrix}$$

The function can be represented as:

$$\begin{vmatrix} | & \dots & | \\ y_1 & & y_n \\ | & \dots & | \end{vmatrix} = \begin{vmatrix} a_{1,1} & \dots & a_{1,d} & b_1 \\ \vdots & \ddots & \vdots & \vdots \\ a_{1,d} & \dots & a_{d,d} & b_d \end{vmatrix} \begin{vmatrix} | & \dots & | \\ x_1 & & x_n \\ | & & | \\ 1 & \dots & 1 \end{vmatrix}$$

So, we have a closed solution for Θ:

$$\Theta = (XX^T)^{-1}(XY^T)$$

2.3.3 Polynomial regression

Another type of linear regression is the polynomial regression, a regression that is linear on the paramenters but polynomial in respect to the data:

$$y_i = \sum_{j=1}^{k} a_j x_i^j + b$$

That in matrix notation is:

$$\begin{vmatrix} y_1 \\ \vdots \\ y_n \end{vmatrix} = \begin{vmatrix} x_1^k & x_1^{k-1} & \dots & x_1 & 1 \\ & & & \vdots & \\ x_n^k & x_n^{k-1} & \dots & x_n & 1 \end{vmatrix} \begin{vmatrix} a_k \\ \vdots \\ a_1 \\ b \end{vmatrix}$$

For the Stone-Weierstrass theorem:
If a function f is continuous on the interval $[a, b]$, then there exists a polynomial p such that $|f(x) - p(x)| = 0$.
We have a closed formula but only if we use the loss function as the MSE, for different loss we don't have a coled formula for minimizing the loss.

2.4 Logistic regression

We want to change the problem into a classification one, in particular a single class problem, in which the output of the model can be only 0 or 1.
So, the classification is not a continuous function.
The loss function is now not convex in a, b:

$$\ell_\Theta(\{x_i, y_i\}) = \sum_{i=1}^{n} (y_i - \sigma(ax_i + b))^2$$

In which σ is the logistic sigmoid defined as:

$$\sigma(x) = \frac{1}{1 - e^{-x}} \implies \sigma(ax_i + b) = \frac{1}{1 - e^{-(ax_i+b)}}$$

We can change the loss into two cases, making it convex:

$$\ell_\Theta(\{x_i, y_i\}) = \sum_{i=1}^{n} c(x_i, y_i) \text{ where } c(x_i, y_i) = \begin{cases} -\ln(\sigma(ax_i + b)) & y_i = 1 \\ -\ln(1 - \sigma(ax_i + b)) & y_i = 0 \end{cases}$$

This cases can be merged into a single expression:

$$c(x_i, y_i) = -y_i \ln(\sigma(ax_i + b)) - (1 - y_i) \ln(1 - \sigma(ax_i + b))$$

And the gradient is:

$$\nabla_\Theta \ell_\Theta = \nabla_\Theta \sum_{i=1}^{n} y_i \ln(\sigma(ax_i + b)) + (1 - y_i) \ln(1 - \sigma(ax_i + b)) = 0$$

To solve this we need to calculate the gradient in respect to a and to b respectively. To do that we have to calculate the derivatives of the two logarithms:

$$\nabla_\Theta \ell_\Theta = y_i \nabla_\Theta \ln(\sigma(ax_i + b)) + (1 - y_i) \nabla_\Theta \ln(1 - \sigma(ax_i + b)) = 0$$

For example we will calculate the derivatives of only $\ln(\sigma(ax_i + b))$ in respect to a applying the chain rule:

$$\frac{\partial}{\partial a} \ln(\sigma(ax_i + b)) = \frac{\partial \ln(\sigma(ax_i + b))}{\partial \sigma(ax_i + b)} \cdot \frac{\partial \sigma(ax_i + b)}{\partial (ax_i + b)} \cdot \frac{\partial (ax_i + b)}{\partial a}$$

The result of the partial derivatives is:

$$\frac{\partial}{\partial a} \ln(\sigma(ax_i + b)) = (1 - \sigma(ax_i + b))x_i$$

2.5 Gradient descent

Gradient descent is a firts-order iterative minimization algorithm. Given a loss function ℓ_Θ the algorithm works as follows:

1. Start from a point Θ^0

2. Iteratively compute:
$$\Theta^{t+1} = \Theta^t - \alpha \nabla \ell_{\Theta^t}$$
where α is called the learning rate.

The loss function ℓ_Θ must be differentiable at all points.
The gradient descent gets stuck on stationary points, points where the gradient of the loss is 0 but the point is not a global minimum or a local minimum.
The learning rate can be adaptive or follow a schedule, some example are:

- Decrease α following a parameter ρ, in different ways:
$$\alpha^{t+1} = \left(1 - \frac{t}{\rho}\right)\alpha^0 + \frac{t}{\rho}\alpha^\rho$$
$$\alpha^{t+1} = \frac{\alpha^t}{1+\rho t}$$
$$\alpha^{t+1} = \alpha^0 e^{-\rho t}$$
- Accumulate past gradients and keep moving in their direction (momentum):
$$v^{t+1} = \lambda v^t - \alpha \nabla \ell_{\Theta^t}$$
$$\Theta^{t+1} = \Theta^t + v^{t+1}$$
The steps lenght is proportional to the sequence of gradients.

We can write the formula of gradient descent in a simpler way:
$$\Theta^{t+1} = \Theta^0 - \alpha \sum_{i=0}^{t} \nabla \ell_{\Theta^t}$$
With the momentum the formula becomes:
$$\Theta^{t+1} = \Theta^0 - \alpha \sum_{i=0}^{t} \frac{1-\lambda^{t+1-i}}{1-\lambda} \nabla \ell_{\Theta^t}$$
This two formulas can be generalized for some γ_i or diagonal matrix Γ_i:
$$\Theta^{t+1} = \Theta^0 + \alpha \sum_{i=0}^{t} \gamma_i^t \nabla \ell_{\Theta^t} = \Theta^0 + \alpha \sum_{i=0}^{t} \Gamma_i^t \nabla \ell_{\Theta^t}$$

2.5.1 Stochastic gradient descent

In Deep Learning each parameter is updated to decrease the loss:
$$\theta_i = \theta_i - \alpha \frac{\partial \ell}{\partial \theta_i}$$
Calculate the loss for each training data point is a huge bottleneck, to mitigate this problem mini-batches are used. Given the training set $\mathcal{T}$ with n data points we can use a mini-batch $\mathcal{B}$ with $m < n$ data points to approximate the gradient:
$$\frac{1}{m}\sum_{i=1}^{m} \nabla \ell_\Theta(\mathcal{B}) \approx \frac{1}{n}\sum_{i=1}^{n} \nabla \ell_\Theta(\mathcal{T})$$
With the mini-batch drawn uniformly.
The stochastic gradient descent algorithm works as follows:

1. Initialize θ.
2. Pick a mini-batch $\mathcal{B}$.
3. Update the parameters:
$$\theta = \theta - \alpha \nabla \ell_\theta(\mathcal{B})$$
4. Repeat from step 2.

When the steps 2-4 cover the entire training set we have an epoch.
This change applied in stochastic gradient descent gives a significant speed-up compared to the normal gradient descent. With n training data point, d parameters and some constants related to the problem k and v, the cost of the two algorithm is:

- Gradient descent:
  - Each iteration: $O(nd)$
  - Iteration to reach the level of accuracy we want ρ: $O\left(k \log\left(\frac{1}{\rho}\right)\right)$
- Stochastic gradient descent:
  - Each iteration: $O(d)$
  - Iteration to reach the level of accuracy we want ρ: $\frac{vk^2}{\rho} + O\left(\frac{1}{\rho}\right)$

2.6 Back-propagation

Consider multiple layers of regression models, defined as:

$$g_\Theta(x) = (\sigma \circ f_{\Theta_n}) \circ \cdots \circ (\sigma \circ f_{\Theta_1})(x)$$

We call it a multi-layer perceptron or deep feed-forward neural network.
Each layer output has an hidden representation:

$$x_{l+1} = \sigma_l(W_l x_l + b_l)$$

where W_l contains the weights and b_l the bias.

Universal Approximation Theorem

For any compact set $\Omega \subset \mathbb{R}^p$, the space spanned by the function $\phi(x) = \sigma(Wx + b)$ is dense in $C(\Omega)$ for the uniform convergence.
For any continous function f there exists a q and weights U such that:

$$\left| f(x) - \sum_{k=1}^{q} u_k \phi_k(x) \right| = 0 \quad \forall x \in \Omega$$

2.6.1 Computational graphs

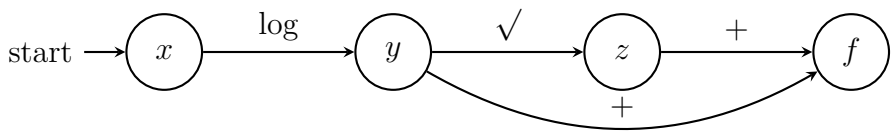
Given a generic function f the computational graph is a directed acyclic graph that represent the compuatation of f.

Example:

Given the function:

$$f(x) = \log x + \sqrt{\log x}$$

The computational graph is:



We can evaluate f by doing a forward traversal of the graph. This is done by calculating the partial derivatives of any intermediate node in respect to x.

1. $\frac{\partial x}{\partial x} = 1$
2. $\frac{\partial y}{\partial x} = \frac{\partial \log x}{\partial x} = \frac{1}{x}$
3. $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x} = \frac{\partial \sqrt{y}}{\partial y}\frac{\partial y}{\partial x} = \frac{1}{2\sqrt{y}}\frac{1}{x}$
4. $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y}\frac{\partial y}{\partial x} + \frac{\partial f}{\partial z}\frac{\partial z}{\partial x} = \frac{\partial (y+z)}{\partial y}\frac{\partial y}{\partial x} + \frac{\partial (y+z)}{\partial z}\frac{\partial z}{\partial x} = \frac{1}{x} + \frac{1}{2\sqrt{y}}\frac{1}{x}$

The cost of doing this is proportional to the number of output, so for a function $f : \mathbb{R}^p \to \mathbb{R}$, the cost will be:

$$\text{compute } \nabla f(x) = p \cdot \text{compute } f(x)$$

To speed-up the process we can evaluate the function in reverse, calculating the derivatives with respect to their inner nodes.

Example:

With the same function as before:

$$f(x) = \log x + \sqrt{\log x}$$

the partial derivatives calculated in reverse are:

1. $\frac{\partial f}{\partial f} = 1$
2. $\frac{\partial f}{\partial z} = \frac{\partial (y+z)}{\partial z} = 1$
3. $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial z}\frac{\partial z}{\partial y} + \frac{\partial f}{\partial y} = \frac{\partial \sqrt{y}}{\partial y} + \frac{\partial (y+z)}{\partial y} = \frac{1}{2\sqrt{y}} + 1$
4. $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial y}\frac{\partial y}{\partial x} = \frac{\partial f}{\partial y}\frac{\partial \log x}{\partial x} = \left(\frac{1}{2\sqrt{y}} + 1\right)\frac{1}{x} = \frac{1}{2\sqrt{y}}\frac{1}{x} + \frac{1}{x}$

The automatic differentation computing in reverse needs two steps:

1. Forward evaluation of the interior nodes, only to calculate values (no derivatives).

2. Backward pass to compute derivatives.

With loss function that always are in the form:

$$\ell : \mathbb{R}^p \to \mathbb{R}$$

where p is very high, the cost of the two approches is very different:

- Forward:
$$p \cdot \sum_{k=2}^{t-1} d_k d_{k+1}$$
- Backward:
$$\sum_{k=1}^{t-2} d_k d_{k+1}$$

The backward approach applied to deep neural network, called **back-propagation**, has a cost that is p times faster.